Típus és osztály

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

Procedurális vs. objektumelvű programozás

- Procedurális programozás: egy probléma részfeladatait megoldó tevékenységeket önálló egységekbe, ún. procedurákba (részprogram, makró, eljárás, függvény) szervezzük. A problémát megoldó folyamatot ezen procedurák közötti vezérlés-átadások (eljárások, függvények esetében hívások) láncolata mutatja meg.
- Objektumelvű programozás: egy probléma megoldáshoz szükséges adatok egy-egy részét a hozzájuk kapcsolódó tevékenységekkel (az ún. metódusokkal) együtt egységekbe, ún. objektumokba zárjuk. A problémát megoldó folyamatot ezen objektumok metódusai közötti vezérlés-átadások (hívások vagy szignálok) jelöli ki.

Feladat

Egy számsorozatban 0 és m közé eső természetes számok találhatók. Melyik a sorozat leggyakoribb eleme?

- Procedurális megoldás: maximum kiválasztás és számlálás
  - Megszámoljuk a számsorozat elemeinek a számsorozatbeli előfordulásainak számát, és ezek közül megkeressük a legnagyobbat.
- Objektumelvű megoldás: tároló objektum
  - Elhelyezzük a tároló objektumban a számsorozat elemeit, majd lekérdezzük a benne található leggyakoribb elemet. Legyen az elhelyezés is, és a lekérdezés is gyors.

Elemzés

Ezek a feladat (input, output) változói, amelyek a megoldást adó program változói is lesznek.

$A = (m:\mathbb{N} , x:\mathbb{N}^*, elem:\mathbb{N})$

m kezdőértéke: $m' \in \mathbb{N}$
x kezdőértéke: $x' \in \mathbb{N}^*$

x' legalább 1 hosszú sorozat, x' elemei 0 és m' közé esnek

$Ef = (m = m' \wedge x = x' \wedge |x| \geq 1 \wedge \forall i \in [1 .. |x|]: x_i \in [0 .. m])$

A változók értékét jellemzi, még a megoldó program elindulása előtt.

Dobáljuk be x' elemeit a zsákba, majd kérjük le annak a leggyakoribb elemét.

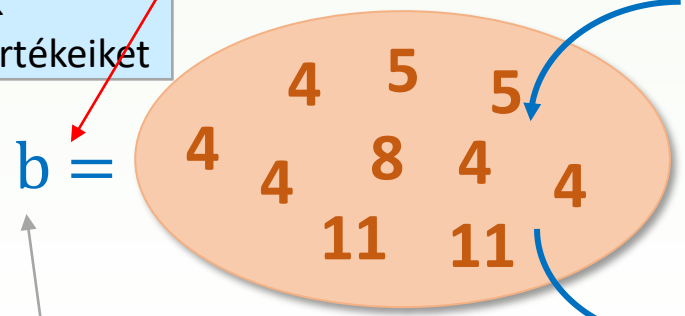
$Uf = (m = m' \wedge x = x' \wedge b:Zsák \wedge b = \bigcup_{i=1}^{|x|} [x_i] \wedge elem = leggyakoribb(b))$

A változók értékét jellemzi, már a megoldó program leállása után.

az input-változók megőrzik kezdőértékeiket

b:=b∪[e]

Itt az ∪ egy zsákot egy zsákkal egyesítő homogén binér művelet, amelynek neutrális eleme az üres zsák (∅).
Az [e] pedig az e-t tartalmazó egyelemű zsákot jelöli.



Zsák típusú segédváltozó

e:=leggyakoribb(b)

Tervezés

$A = (m:\mathbb{N} , x:\mathbb{N}^*, elem:\mathbb{N})$

$Ef = (m = m' \wedge x = x' \wedge |x| \geq 1 \wedge \forall i \in [1 .. |x|]: x_i \in [0 .. m])$

Dobáljuk be x' elemeit a zsákba, majd kérjük le a zsák leggyakoribb elemét.

$Uf = (m = m' \wedge x = x' \wedge b:Zsák \wedge b = \overline{\bigcup}_{i=1}^{|x|} [x_i] \wedge elem = leggyakoribb(b))$

végrehajtható specifikáció: elmosódik az elemzés és a tervezés közötti határ (mi a feladat és hogyan oldjuk meg)

s : H
f:[m..n]⟶H
+:H×H⟶H
neutrális elem a 0

A bezsákolás egy speciális összegzés:

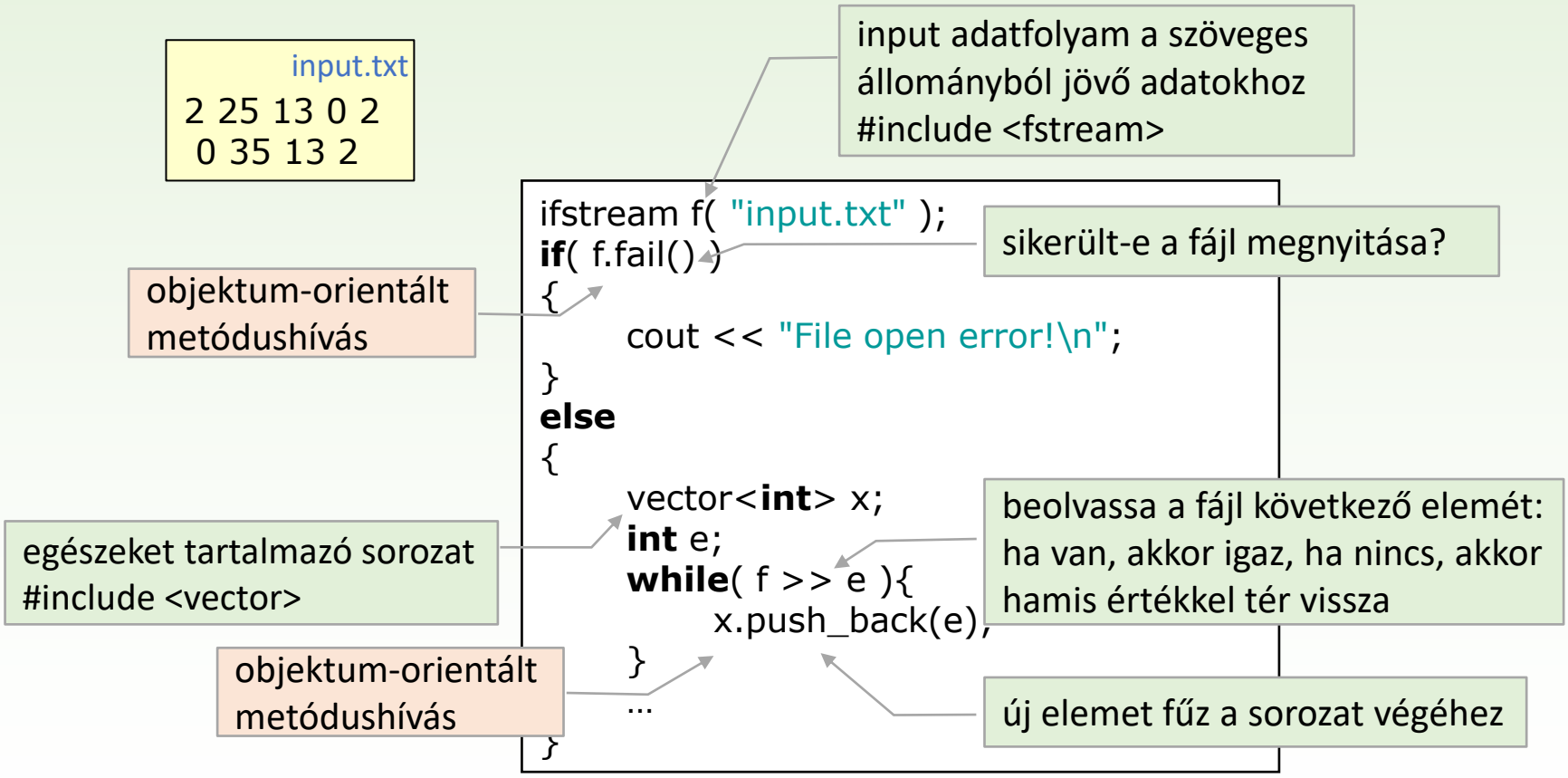
$$s = \sum_{i=m}^{n} f(i) \quad \sim \quad b = \overline{\bigcup}_{i=1}^{|x|} [x_i]$$

eredmény:	s : H	~	b : Zsák
művelet:	H, +, 0	~	Zsák, $\overline{\cup}$, Ø
elem:	f(i)	~	$[x_i]$
felsorolás:	i = m .. n	~	i = 1 .. |x|

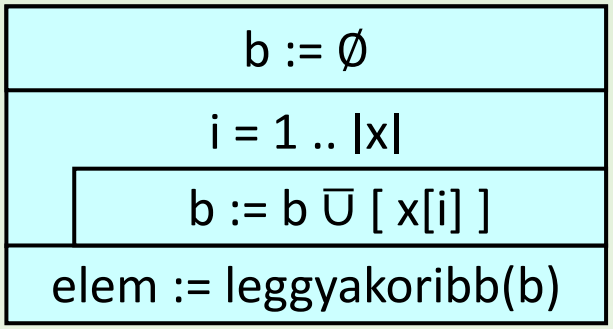
```
b := Ø
i = 1 .. |x|
    b := b ∪̄ [x_i]
elem := leggyakoribb(b)
```

Megvalósítás: kitérő

Hogyan töltünk fel C++ programozási nyelven egy sorozatot egy szöveges állományból beolvasott adatokkal?



Megvalósítás



m
x

input.txt
35
2 25 13 0 2
0 35 13 2

```
int main()
{
	ifstream f( "input.txt" );
	if( f.fail() ){
		cout << "File open error!\n"; return 1;
	}
	int m; f >> m;
	Bag b(m);
	b.erase();
	int e;
	while( f >> e ){
		b.putIn(e);
	}
	cout << "The most frequent number: " << b.mostFrequent() << endl;

	return 0;
}
```

megspóroljuk az else ágat

Zsák

b := Ø

Nem építjük fel az x sorozatot csak azért, hogy majd aztán ebből töltsük fel a zsákot, hanem közvetlenül pakoljuk az elemeket a fájlból a zsákba.

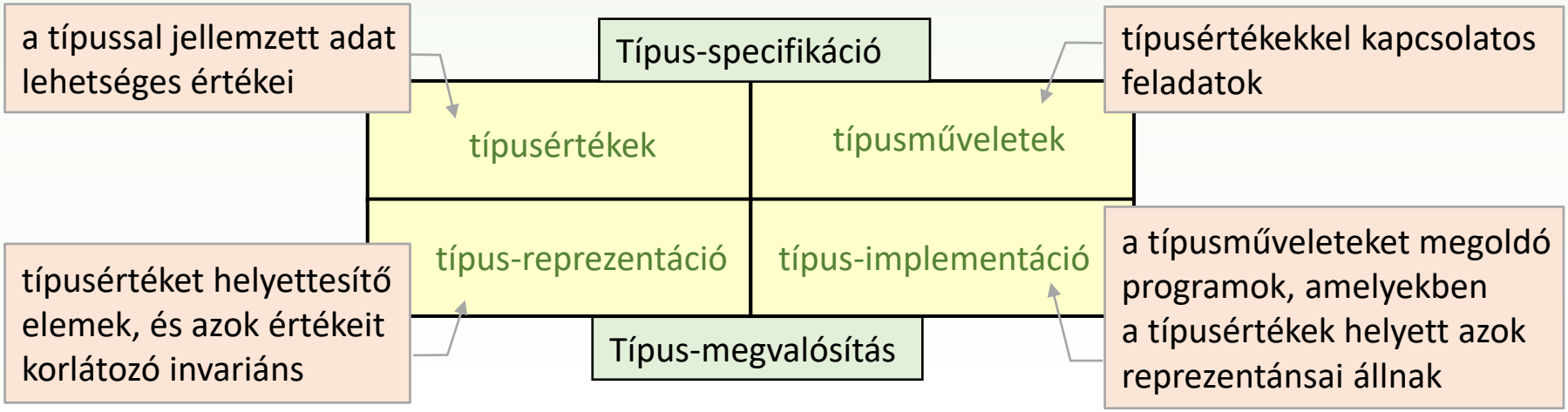
b := b ∪ [e]

leggyakoribb(b)

Hogyan adjuk meg a Bag és a műveleteinek a jelentését? Ehhez a Zsák típust kell megtervezni, majd kódolni.

Adattípus fogalma

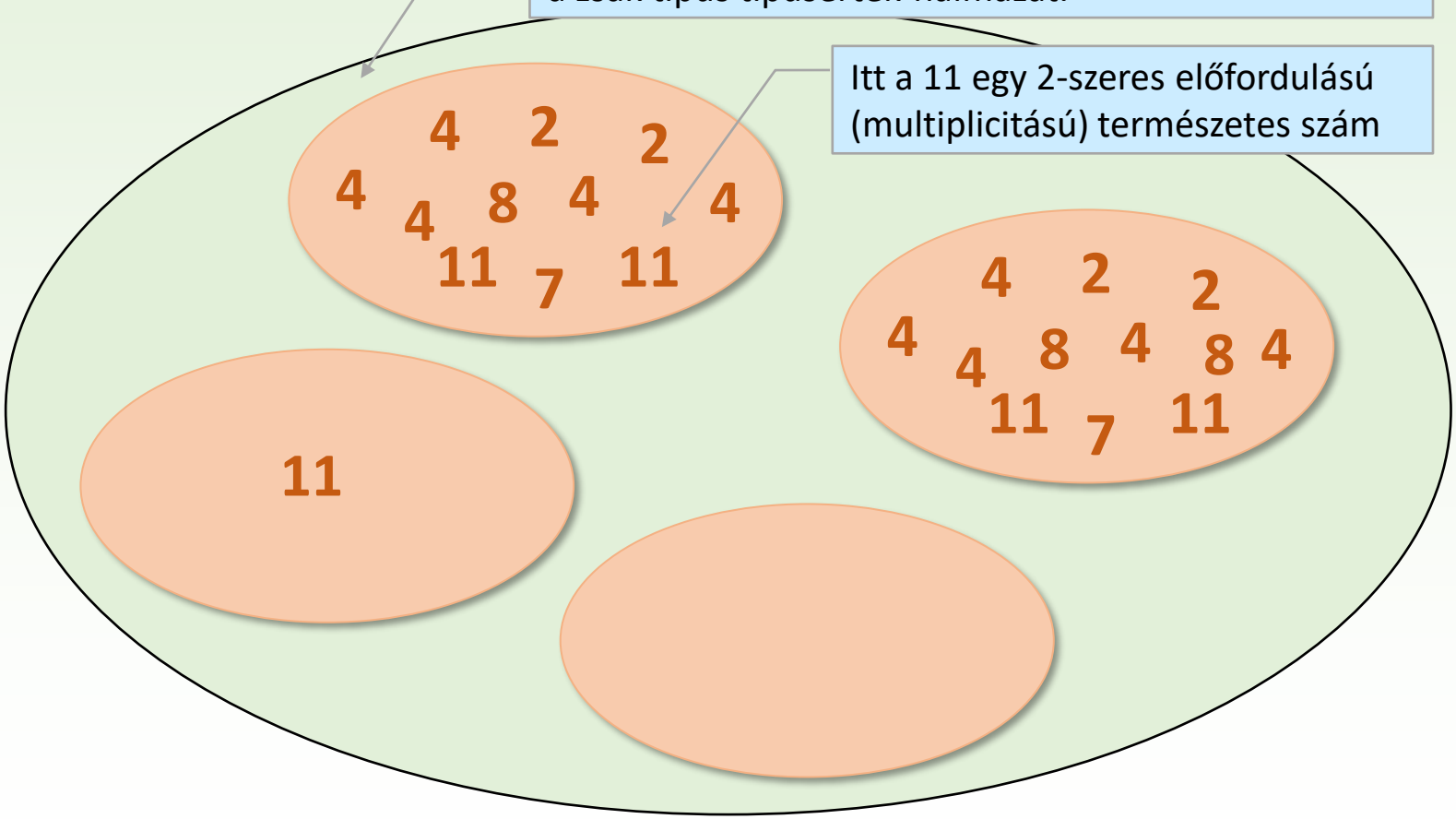
- Egy adat (változó) típusának definiálásához szükség van a típus specifikációjára és annak megvalósítására.
- A típus-specifikáció megadja:
  - az adat által felvehető értékek halmazát
  - a típusértékekkel végezhető műveleteket
- A típus-megvalósítás megmutatja:
  - hogyan ábrázoljuk (reprezentáljuk) a típus értékeit
  - milyen programok helyettesítsék (implementálják) a műveleteket



Zsák típus típusérték-halmaza

Egy természetes számokat tároló zsák típusú adatnak (változónak) az értékei (típusértékei) természetes számokat tároló zsákok. Az összes ilyen zsák alkotja a zsák típus típusérték-halmazát.

Itt a 11 egy 2-szeres előfordulású (multiplicitású) természetes szám



Zsák típus műveletei

Kiüríti a zsákot:

b := ∅ b:Zsák

adjon hibajelzést, ha $e \notin [0 .. m]$

Betesz egy elemet a zsákba:

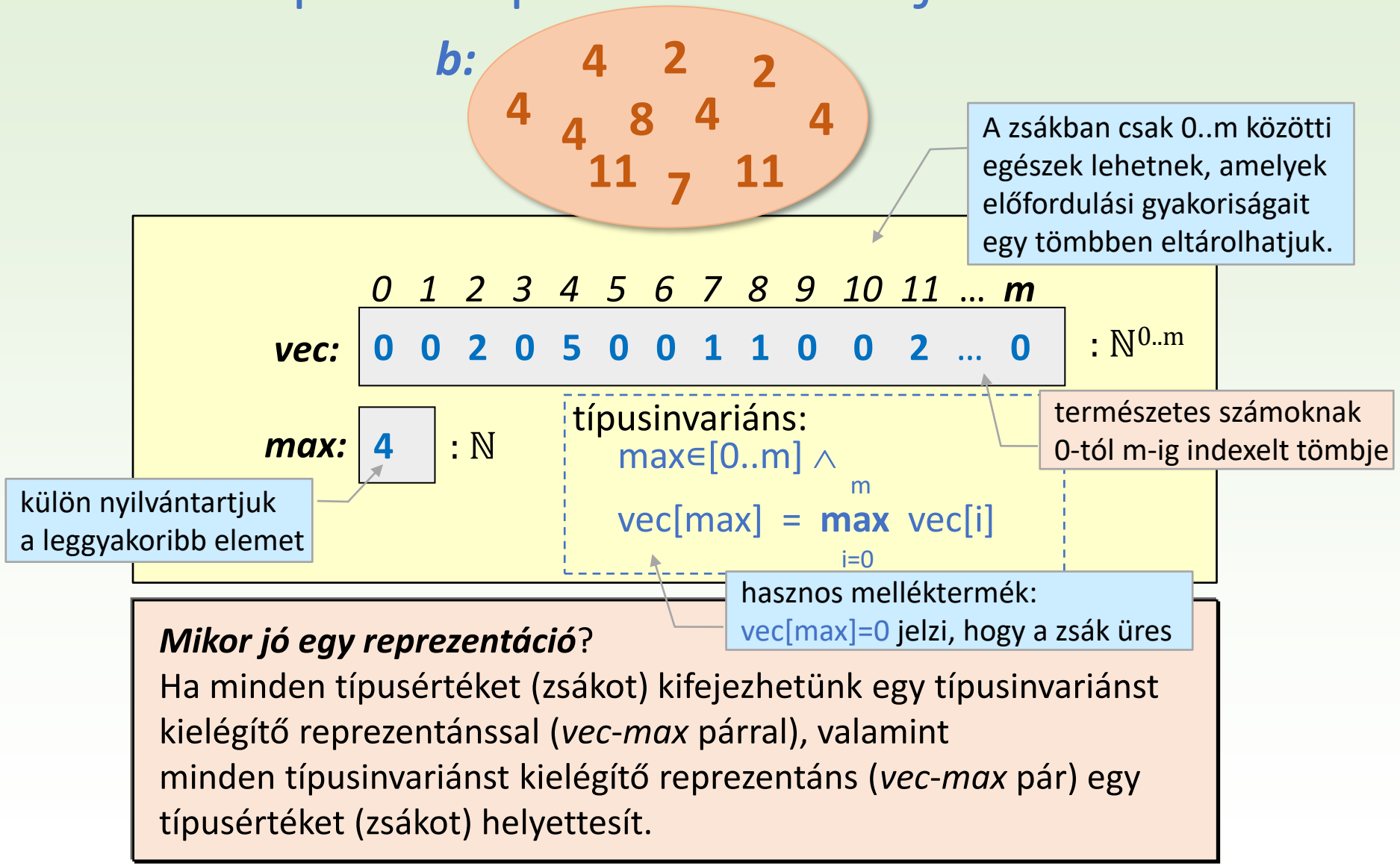
b := b ⊍ [e] b:Zsák , e:ℕ

adjon hibajelzést, ha a zsák üres

Zsák leggyakoribb eleme:

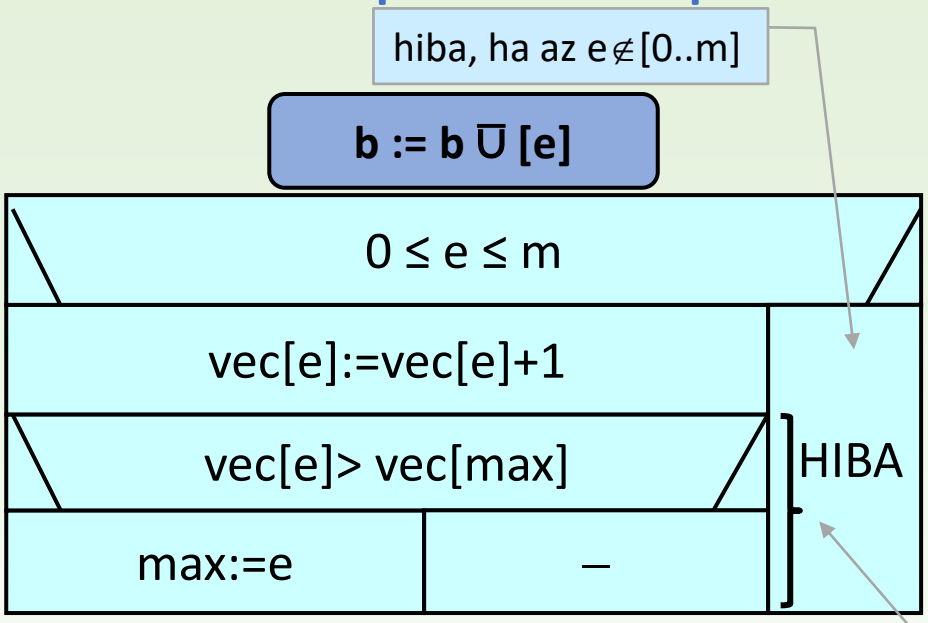
e:=leggyakoribb(b) b:Zsák , e:ℕ

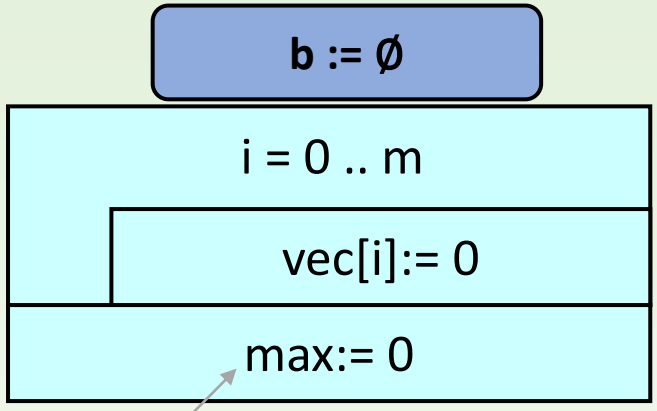
Zsák típus reprezentációja



Mikor jó egy reprezentáció?
Ha minden típusértéket (zsákot) kifejezhetünk egy típusinvariánst kielégítő reprezentánssal (*vec-max* párral), valamint
minden típusinvariánst kielégítő reprezentáns (*vec-max* pár) egy típusértéket (zsákot) helyettesít.

Zsák típus implementációja



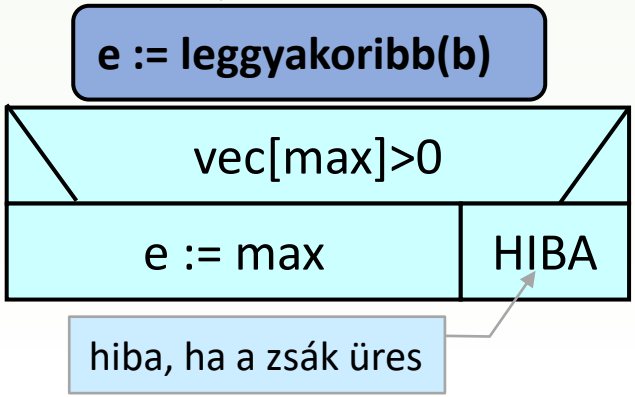


a típusinvariáns biztosításához a max a 0..m bármelyik eleme lehet, mert ∀i∈[0..m]:vec[i]=0

típusinvariáns miatt kell

nem kell ellenőrizni a típusinvariánst

Mikor jó az implementáció?
Ha minden típusműveletet egy olyan program helyettesít, amelyben a típusértékeket (zsákokat) a típusinvariánst kielégítő reprezentánsaik (*vec*, *max* adattagok) helyettesítik.



típusinvariáns:

$$max \in [0..m] \wedge vec[max] = \max_{i=0}^{m} vec[i]$$

Objektum fogalma

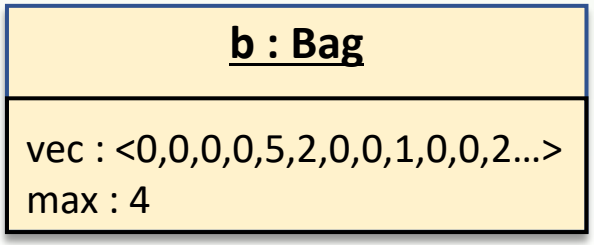
például a b zsák (Bag)

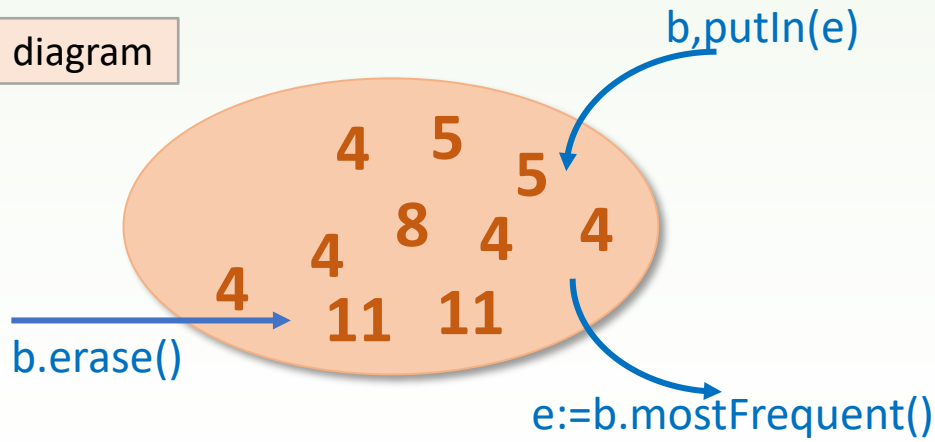
- Az objektum egy olyan önálló egyedként azonosított része a feladat megoldásának, amely a megoldás egy részéért felelős adatokat, és az ezekkel kapcsolatos műveleteket foglalja magába.

vec, max

kiürít, betesz, leggyakoribb

UML objektum diagram





Osztály fogalma

- Az osztály egy objektum szerkezetének és viselkedésének a mintáját adja meg, azaz
  - felsorolja az objektum adattagjait a láthatóságuk, nevük, és típusuk megadásával
  - megadja az objektumra meghívható metódusokat (tagfüggvény, művelet) a láthatóságuk, nevük, paraméterlistájuk, visszatérési értékük típusa, és törzsük megadásával
- Az osztály lényegében az objektum típusa: az objektumot az osztálya alapján hozzuk létre, azaz példányosítjuk.
- Egy osztályhoz több objektum is példányosítható: minden objektum rendelkezik az osztályleírás által leírt adattagokkal és metódusokkal.

vec : int[0..m], max : int

Erase() : void
PutIn(int) : void
MostFrequent() : int

Bag
- vec : int[0..m] - max : int
+ Erase() : void + PutIn(e:int) : void + MostFrequent() : int

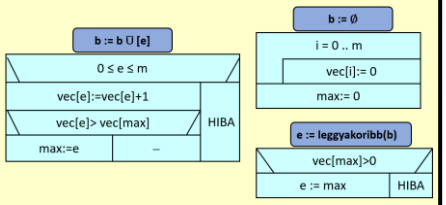
UML osztály diagram

Típus és Osztály

- Az objektumelvű tervezés során osztályként adjuk meg az egyedi, vagy ún. felhasználói típusokat (*custom type*).

Zsák típus:

típus-specifikáció

típus-értékek	típus-műveletek
zsákok	b := ∅ b := b ∪ [e] e:=leggyakoribb(b) b:Zsák, e:ℕ
reprezentáció	**implementáció**
vec : $\mathbb{N}^{0..m}$ max : ℕ { max∈[0..m] ∧ vec[max] = **max**$_{i=0..m}$ vec[i] }	

típus-megvalósítás

Zsák osztály:

max∈[0..m] ∧
vec[max] = **max**$_{i=0..m}$ vec[i]

Bag

- vec : int[0..m]
- max : int

+ Erase() : void
+ PutIn(e:int) : void
+ MostFrequent() : int

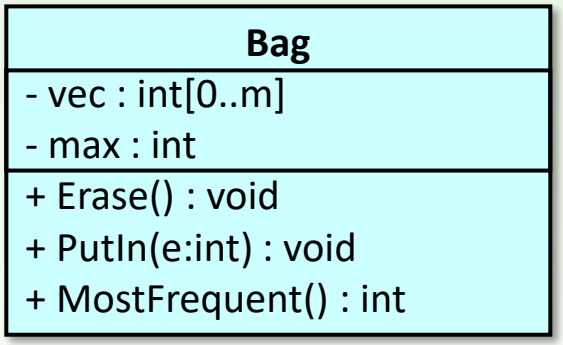
```
for i=0..m loop vec[i] := 0 endloop
max := 0
```

```
if vec[max]>0 then return max
else error
endif
```

```
if 0≤e≤m then
    ++vec[i]
    if vec[e]>vec[max] then  max := e endif
else error
endif
```

Zsák típus osztálya C++ nyelven

- A C++ nyelvben az osztályt a *class* nyelvi elem segítségével írjuk le.



típus neve

```
class Bag
{
    private:
        vector<int> _vec;
        int _max;

    public:
        void erase();
        void putIn(int e);
        int  mostFrequent() const;
};
```

adattagok

metódusok

Ez egy konstans metódus, amely nem változtatja meg az objektum adattagjainak értékét.

1

Az objektum-orientált nyelvek lényeges ismérve az egységbezárás: egy adott feladatkör megvalósításához szükséges adatokat és az azokat manipuláló programrészeket a program többi részétől elkülönítve adhatjuk meg.

Zsák típus metódusai C++ nyelven

a Bag osztályhoz tartozó

sorozat hossza: unsigned int típusú

castolás int-re

```
void Bag::erase()
{
      for(unsigned int i=0; i<_vec.size(); ++i) _vec[i] = 0;
      _max = 0;
}

void Bag::putIn(int e)
{
      if( e<0 || e>=int(_vec.size()) ) return; // hibakezelés még hiányzik
      if( ++_vec[e] > _vec[_max] ) _max = e;
}

int Bag::mostFrequent() const
{
      if( 0 ==_vec[_max] ) return -1; // hibakezelés még hiányzik
      return _max;
}
```

A vec és a max adattagok ahhoz a zsák objektumhoz tartoznak, amelyikkel az őket tartalmazó a metódust meghívják.

Konstruktor

Egy objektum példányosítását (létrehozását) egy speciális metódus, a konstruktor végzi. Ennek feladata, hogy létrehozza (lefoglalja a memóriában) az objektum adattagjait, és ehhez az adattagok saját konstruktorait használja.

Amíg mást konstruktort nem definiálunk, addig rendelkezünk egy (paraméter és törzs nélküli) ún. üres konstruktorral.

nincs visszatérési típusa
neve: az osztályának neve

```
class Bag
{
    private:
        vector<int> _vec;
        int _max;
    public:
        Bag(int m);
        ...
};
```

A Bag b deklaráció vagy Bag *b = **new** Bag() utasítás az üres konstruktort hívja, amely egy nulla hosszú _vec nevű sorozatot, és egy _max nevű integert hoz létre.

De nekünk olyan konstruktor kell, amelynek átadhatjuk a zsákba kerülő számok maximumát a _vec hosszaként (pl. Bag b(35)). Ráadásul a zsák üres lesz, ha ennek során a _vec sorozat elemeit és a _max értékét nullára állítjuk (úgy mint az erase()-ben).

Alternatívák a konstruktorra:

Létrehozza az adattagokat azok üres konstruktorainak implicit módon történő hívásaival, majd végrehajtja a törzse utasításait.

```
Bag::Bag(int m) { _vec.resize(m+1); erase(); }
```

```
Bag::Bag(int m) { _vec.resize(m+1, 0); _max = 0; }
```

A vector átméretezését végző metódusnak több alakja van: megadható a sorozatot feltöltő érték.

Itt explicit módon hívjuk meg az adattagok konstruktorait.

```
Bag::Bag(int m) : _vec(m+1, 0), _max(0) { }
```

Hivatkozás egy objektum tagjaira

```
class Bag
{
    private:
        vector<int> _vec;
        int _max;
    public:
        Bag(int m);
        void erase();
        void putIn(int e);
        int  mostFrequent() const;
};
int main()
{
    Bag b1, b2;
    b1.erase();
    b2.putIn(5);
    int a = b2.mostFrequent();
    b1._max = 0;
    b1._vec[5]++;
}
```

objektumok példányosítása

Amikor egy objektum egy tagjára (adattagra vagy metódusra) hivatkozunk, akkor magát az objektumot (pontosabban az arra hivatkozó változót) odaírjuk a tag elé.
A metódus elé írt objektum extra paramétere (input- és output adata) a metódusnak.
(Konstans metódusnál csak input adat.)

Egy objektum rejtett (privát, védett) tagjait csak az objektum metódusainak definíciójában (azok törzsében) használhatjuk, máshol ezeket nem használhatjuk (nem hivatkozhatunk rájuk)

2

Az objektum orientált nyelvek fontos ismérve az elrejtés: az egységbe zárt elemek láthatóságát korlátozhatjuk. (Általában az adattagok rejtettek, azok értékéhez csak közvetetten, a publikus metódusokkal férünk hozzá.)

Zsák osztály fejállományban

projekt:
- main.cpp
- bag.h
- bag.cpp

fejállományok elejére, hogy kizárjuk a többszörös bemásolásokat (nem szabványos)

```
#pragma once

#include <vector>

class Bag
{
        class NegativeSizeException   : std::exception {};
        class IllegalElementException : std::exception {};
        class EmptyBagException       : std::exception {};

private:
        std::vector<int> _vec;
        int _max;
public:
        Bag(int m);
        void erase();
        void putIn(int e);
        int  mostFrequent() const;
};
```

hiba esetek kivételként való megadása felsorolása

A fejállomány elejére nem szokás beírni a *using namespace std*-t, ezért külön jelezzük, hogy a vector definíciója az std névtérben található.

bag.h

Külön fejállomány (header fájl), amelyet minden olyan forrásfájlba „beinklúdolunk”, ahol a benne levő definíciókat használni akarjuk.

Metódusok külön egységben

projekt:
main.cpp
bag.h
bag.cpp

Bag osztály definíciójának bemásolása

```
#include "bag.h"

Bag::Bag(int m) : _vec(m+1, 0), _max(0)
{
	if( m<0 ) throw NegativeSizeException();
}
void Bag::erase()
{
	for(unsigned int i=0; i<_vec.size(); ++i) _vec[i] = 0;
	_max = 0;
}
void Bag::putIn(int e)
{
	if( e<0 || e>=int(_vec.size()) ) throw IllegalElementException();
	if( ++_vec[e] > _vec[_max] ) _max = e;
}
int Bag::mostFrequent() const
{
	if( 0 ==_vec[_max] ) throw EmptyBagException();
	return _max;
}
```

kivétel-dobás: a hiba észlelésekor még nem kezeljük le a hibát, csak jelezzük.

kivételt dob, ha m értéke negatív

kivételt dob, ha a paraméter értéke nincs 0 és m között

kivételt dob, ha b üres

külön fordítási egység

bag.cpp

Főprogram

projekt:
- main.cpp
- bag.h
- bag.cpp

```
#include <iostream>
#include <fstream>
#include "bag.h"
using namespace std;

int main()
{
    ifstream f( "input.txt" );
    if( f.fail() ){   cout << "File open error!\n";  return 1; }
    int m; f >> m;
    if( m<0 ){ cout << "Upper limit of natural numbers cannot be negative!\n"; return 1; }
    try {
        Bag b(m);
        int e;
        while( f >> e ) {  b.putIn(e); }
        cout << "The most frequent element: "<< b.mostFrequent() << endl;
    } catch(Bag::NegativeSizeException) {
        cout << "Upper limit of the natural numbers is negative.\n";
    } catch(Bag::IllegalElementException) {
        cout << "One of the given elements is not in the interval 0 .. m.\n";
    } catch(Bag::EmptyBagException) {
        cout << "There is no most frequented number if there are no numbers.\n";
    }
    return 0;
}
```

Bag osztály definíció bemásolása

kivételek figyelése

kivételek kezelése

main.cpp

https://github.com/catchorg/Catch2/blob/master/docs/tutorial.md

Automatikus tesztelés

```
#include <iostream>
#include <fstream>
#include "bag.h"
using namespace std;


#define CATCH_CONFIG_MAIN
#include "catch.hpp"

TEST_CASE("empty sequence", "[main]"){
      ifstream f( "input1.txt" ); REQUIRE(!f.fail());
      int m; f >> m; CHECK(m==35); REQUIRE(m>=0);
      Bag b(m);
      int e;
      while( f >> e) { b.putIn(e); }
      CHECK _THROWS(b.mostFrequent());
}
TEST_CASE("one-length sequence", "[main]"){
      ifstream f( "input2.txt" ); REQUIRE(!f.fail());
      int m; f >> m; CHECK(m==35); REQUIRE(m>=0);
      Bag b(m);
      int e;
      while( f >> e ){ b.putIn(e); }
      CHECK(b.mostFrequent()==2);
}
...
```

folytatódik a teszteset vizsgálata, ha a zárójelbeli feltétel hamis

megszakad a teszteset vizsgálata, ha a zárójelbeli feltétel hamis

input1.txt = < 35 > (m = 35)
b = [ ] → error: No most frequented element

kivételek keletkezése is tesztelhetők

input2.txt = < 35, 2 > (m = 35)
b = [2(1)] → The most frequented element = 2

main.cpp

https://github.com/catchorg/Catch2/blob/master/docs/tutorial.md

Automatikus egység (unit) tesztek

```
TEST_CASE("creation of an empty bag", "[bag]")
{
      int m = 0;
      Bag b(m);
      vector<int> v = { 0 };
      CHECK( v == b.getArray() );
}
TEST_CASE("new element into empty bag", "[putIn]")
{
      Bag b(1);
      b.putIn(0);
      vector<int> v = { 1, 0 };
      CHECK( v == b.getArray() );
}
```

üres zsák létrehozása:
m = 0 → _vec = < 0 >

Mivel a b._vec privát, azt nem tudjuk közvetlenül lekérdezni.

üres zsákba új elem:
m = 1, e = 0 → _vec = < 1, 0 >

```
class Bag {
private:
      std::vector<int> _vec;
      int _max;
public:
      …
      const std::vector<int>& getArray() const {return _vec;}
};
```

„inline” definíció